Abaixo está um plano completo, estruturado para execução manual (ou por outro agente), garantindo **POC com Behave + pytest-behave**, sem modificar o código existente e maximizando reaproveitamento.

---

1. PLANO DETALHADO

1.1 Estratégia Geral da PoC

Objetivo da PoC:

- Reutilizar:
  - pages/
  - fixtures/
  - data/
  - utils/
- Substituir:
  - transactions/ → por **steps Behave**
- Manter:
  - specs/ intacto (coexistência)
- Integrar:
  - Behave com Pytest via pytest-bdd ou pytest-behave (neste caso seguir com **pytest-behave** conforme requisito)

1.2 Estrutura de Pastas (NOVA)

Decisão: Onde criar features/?

Recomendação para PoC:

```
tests/
├── features/
│   ├── login/
│   │   ├── login.feature
│   │   ├── steps/
│   │   │   └── login_steps.py
│   ├── checkout/
│   │   ├── checkout.feature
│   │   ├── steps/
│   │    │   └── checkout_steps.py
│   └── environment.py
```

Justificativa

- Mantém isolamento dentro de tests/
- Facilita coexistência com specs/
- Evita poluir raiz do projeto
- Segue padrão de projetos Python multi-test framework

1.3 Arquivos NOVOS a serem criados

Estrutura completa

```
tests/features/
│
├── environment.py
│
├── login/
│   ├── login.feature
│   └── steps/
│       └── login_steps.py
│
├── checkout/
│   ├── checkout.feature
│   └── steps/
│       └── checkout_steps.py
│
tests/docs/
└── behave_poc.md
```

1.3.1 Arquivo: environment.py

Responsável por lifecycle hooks do Behave:

```python
from tests.fixtures.driver import create_driver
from tests.config.settings import Settings


def before_all(context):
    context.settings = Settings()


def before_scenario(context, scenario):
```

```
    context.driver = create_driver()


def after_scenario(context, scenario):
    if hasattr(context, "driver"):
        context.driver.quit()
```

1.3.2 Arquivo: login.feature

Exemplo:

```
Feature: Login

  Scenario: Login válido
    Given que o usuário acessa a página de login
    When ele realiza login com usuário válido
    Then ele deve ser redirecionado para a página de inventário
```

1.3.3 Arquivo: login_steps.py

Importante: substituir transactions por chamadas diretas às pages.

```
from behave import given, when, then
from tests.pages.login_page import LoginPage
from tests.pages.inventory_page import InventoryPage


@given("que o usuário acessa a página de login")
def step_open_login(context):
    context.login_page = LoginPage(context.driver)
    context.login_page.open()


@when("ele realiza login com usuário válido")
def step_valid_login(context):
    context.login_page.login("standard_user", "secret_sauce")


@then("ele deve ser redirecionado para a página de inventário")
def step_validate_inventory(context):
```

```
19      inventory_page = InventoryPage(context.driver)
20      assert inventory_page.is_loaded()
```

1.3.4 Documento: tests/docs/behave_poc.md

Conteúdo mínimo:

```
1  # Behave PoC
2
3  ## Objetivo
4  Validar uso de BDD com Behave reutilizando Pages.
5
6  ## Estratégia
7  - Não usar Transactions
8  - Steps chamam diretamente Pages
9
10  ## Execução
11  pytest -k behave
12
13  ## Estrutura
14  tests/features/
15
16  ## Limitações
17  - Duplication de lógica (temporária)
18  - Não substitui specs ainda
```

1.4 Configuração do Behave + Pytest

1.4.1 Adicionar dependências

requirements.txt:

```
1  behave
2  pytest-behave
```

1.4.2 Configuração no pyproject.toml

Adicionar:

```
1  [tool.pytest.ini_options]
2  addopts = "--behave"
```

OU (mais controlado):

```
[tool.pytest.ini_options]
markers = [
    "behave: mark behave tests"
]
```

1.4.3 Ajustes no pytest.ini

Adicionar:

```
[pytest]
addopts = -p pytest_behave
```

1.5 Integração Behave + Pytest

Execução:

```
pytest tests/features
```

OU:

```
pytest -k behave
```

1.6 Portabilidade das Fixtures Pytest → Behave

Estado Atual

Você possui:

```
tests/fixtures/
├── driver.py
└── data_fixture.py
```

Problema

Behave não usa fixtures automaticamente.

Solução: Mapear fixtures para context

1.6.1 Driver Fixture

ANTES (pytest):

```
@pytest.fixture
def driver():
```

```
    driver = create_driver()
    yield driver
    driver.quit()
```

DEPOIS (Behave):

```
def before_scenario(context, scenario):
    context.driver = create_driver()

def after_scenario(context, scenario):
    context.driver.quit()
```

1.6.2 Data Fixture

ANTES:

```
@pytest.fixture
def user_data():
    return load_user()
```

DEPOIS:

```
from tests.data.data_loader import load_user

def before_scenario(context, scenario):
    context.user_data = load_user()
```

Regra geral de migração

Pytest Fixture	Behave
scope=session	before_all
scope=function	before_scenario
yield cleanup	after_scenario

1.7 Substituição de Transactions

Estratégia

NÃO usar:

```
transactions/
```

SIM:

- Steps chamam diretamente Page Objects

Exemplo

ANTES:

```
login_transaction.login()
```

DEPOIS:

```
login_page.open()
login_page.fill_user()
login_page.submit()
```

1.8 GitHub Actions (CI)

1.8.1 Atualizar workflow

Adicionar step:

```
- name: Run Behave Tests
  run: pytest tests/features
```

1.8.2 Pipeline completo (exemplo)

```
- name: Install dependencies
  run: pip install -r requirements.txt

- name: Run Pytest Specs
  run: pytest tests/specs

- name: Run Behave (PoC)
  run: pytest tests/features
```

1.9 Estratégia de Convivência (Specs + Behave)

Durante a PoC:

- tests/specs/ → produção atual
- tests/features/ → PoC

Regras:

- NÃO misturar imports
- NÃO compartilhar steps com specs
- Manter isolamento total

1.10 Casos priorizados para PoC

Selecionar testes simples:

- Login
- Checkout happy path

1.11 Critérios de sucesso da PoC

- Execução via Pytest
- Pages reutilizadas
- Hooks funcionando
- Relatórios existentes funcionando
- Nenhuma alteração nos specs atuais

---

2. SUMÁRIO

O plano foi estruturado com base em:

1. Separação segura entre BDD e testes atuais
2. Reuso máximo da arquitetura existente (Pages)
3. Substituição controlada das Transactions por Steps
4. Aproveitamento do pipeline atual (pytest + GitHub Actions)
5. Tradução direta do lifecycle do Pytest para Behave (hooks)

A abordagem reduz risco e permite comparar:

- legibilidade
- manutenção
- custo de adoção

---

3. BOAS PRÁTICAS PARA PRÓXIMAS ITERAÇÕES

3.1 Evitar duplicação futura

- Criar camada intermediária:
- (entre Page e Step)

3.2 Padronizar Steps

- Steps devem ser:
  - declarativos
  - sem lógica complexa
  - reutilizáveis

Evitar:

```
@login com vários parâmetros diferentes replicados
```

3.3 Tagging estratégico

Usar tags:

```
@smoke
@regression
@login
```

3.4 Data-driven via Gherkin

Exemplo:

```
Scenario Outline: Login
  Given que o usuário acessa login
  When faz login com "<user>" e "<password>"
  Then deve ver "<resultado>"

Examples:
  | user | password | resultado |
```

3.5 Evolução recomendada (após PoC)

- Avaliar substituir Transactions definitivamente
- Criar DSL de testes
- Unificar pytest + BDD ou separar pipelines

---

4. PERGUNTA (BLOQUEANTE)

Para melhorar a precisão do plano:

Você deseja que:

A) Steps reutilizem logicamente as **Transactions existentes (sem modificar código)**
ou
B) Steps ignorem completamente Transactions e usem apenas Pages (como propus)?

Essa decisão muda bastante o design da PoC e o esforço de manutenção.